

Backend Test Report

Small Business Operations Platform

Team	Claudius (Group 61)
Project	Small Business Operations Platform
Domain	Small Retail Clothing Stores (Tier 2/3/4)
Testing Framework	Jest 29 + ts-jest + Supertest
Database	PostgreSQL 16 (smallbiz_test)
Report Date	April 2026

480 / 480 Tests Passing

Table of Contents

1. Executive Summary	3
2. Test Cases — Authentication	3
3. Test Cases — User Management	4
4. Test Cases — Store Configuration	4
5. Test Cases — Categories & Price Bands	5
6. Test Cases — Inventory Batches	5
7. Test Cases — Sales & Returns	6
8. Test Cases — Vendors & Reorders	7
9. Test Cases — Attendance & GPS	8
10. Test Cases — Dashboard, Staff & Audit	8
11. Bugs Found and Fixed During Testing	9
12. User Feedback	11
13. Sprint 2 — Planned Improvements	12
14. Test Infrastructure	13

1. Executive Summary



This report covers the complete integration test suite written for the Small Business Operations Platform backend. All tests run as real HTTP calls against a dedicated PostgreSQL test database. The test suite was written using TDD principles — tests were written first, which directly uncovered 7 critical bugs before deployment.

Suite Breakdown

Test Suite	File	Tests	Status
Authentication	auth.test.ts	27	ALL PASS
User Management	users.test.ts	67	ALL PASS
Store Configuration	store.test.ts	41	ALL PASS
Categories	categories.test.ts	23	ALL PASS
Price Bands	priceBands.test.ts	32	ALL PASS
Inventory Batches	inventory.test.ts	62	ALL PASS
Sales	sales.test.ts	22	ALL PASS
Returns & Exchanges	returns.test.ts	30	ALL PASS
Vendors	vendors.test.ts	45	ALL PASS
Reorders	reorders.test.ts	38	ALL PASS
Attendance & GPS	attendance.test.ts	20	ALL PASS
Staff Leaderboard	staff.test.ts	12	ALL PASS
Dashboard	dashboard.test.ts	15	ALL PASS
Audit Log & Wipe	audit.test.ts	19	ALL PASS
TOTAL		480	480 PASS

2. Test Cases — Authentication

POST /api/auth/register

#	Test Case	Input / Condition	Expected	Result
1	Create store and owner account	Valid shopName, ownerName, email, phone, password, address	201 + token + user	PASS
2	Accept optional logo field	Valid body + logo URL	201 + token	PASS
3	Duplicate email rejected	Email already registered	400	PASS
4	Duplicate shop name rejected	shopName already in use	400	PASS
5	Missing shopName	Body without shopName field	400	PASS
6	Missing ownerName	Body without ownerName	400	PASS
7	Missing email	Body without email	400	PASS
8	Invalid email format	email: "notanemail"	400	PASS
9	Phone not 10 digits	phone: "12345"	400	PASS
10	Phone with non-digit characters	phone: "999999999a"	400	PASS
11	Password shorter than 6 chars	password: "abc"	400	PASS
12	Missing address	Body without address	400	PASS
13	Empty body	{ }	400	PASS

POST /api/auth/login

#	Test Case	Input / Condition	Expected	Result
14	Owner logs in with email + password	Correct credentials	200 + JWT token	PASS
15	Manager logs in with email + password	Correct credentials	200 + JWT token	PASS
16	Staff logs in with phone + PIN	Correct phone + 4-digit PIN	200 + JWT token	PASS

#	Test Case	Input / Condition	Expected	Result
17	Token is a valid JWT	Any successful login	3-part dot-separated string	PASS
18	Wrong password rejected	Correct email, wrong password	401	PASS
19	Wrong PIN rejected	Correct phone, wrong PIN	401	PASS
20	Non-existent email rejected	Unknown email address	401	PASS
21	Non-existent phone rejected	Unknown phone number	401	PASS
22	Missing identifier	No email or phone in body	400	PASS
23	Missing credential	Email present, no password	400	PASS
24	Empty body	{ }	400	PASS
25	Empty string identifier	email: ""	400	PASS
26	Empty string credential	password: ""	400	PASS
27	Staff login via email fails gracefully	Staff has no password_hash	401	PASS

3. Test Cases — User Management

#	Test Case	Role	Expected	Result
1	List users — owner receives full array	Owner	200 + array	PASS
2	Response items have correct shape	Owner	id, name, phone, role, isActive	PASS
3	Manager can list users	Manager	200	PASS
4	Staff can list users	Staff	200	PASS
5	No cross-store user leakage	Owner (store B)	Only own-store users returned	PASS
6	Owner adds staff (phone + PIN)	Owner	201 + user object	PASS
7	Owner adds manager (email + password)	Owner	201 + user object	PASS
8	Manager adds staff	Manager	201	PASS
9	Staff cannot add users	Staff	403	PASS
10	Duplicate phone rejected	Owner	409	PASS
11	Staff PIN must be 4 digits	Owner	400 if PIN length wrong	PASS
12	Manager password required	Owner	400 if password missing	PASS
13	Owner soft-deletes staff	Owner	200 + user gone from GET	PASS
14	Owner cannot delete themselves	Owner	403 protected	PASS
15	Manager cannot delete users	Manager	403	PASS
16	Cross-store update blocked	Owner (store B)	404	PASS
17	Password change requires correct currentPassword	Owner	401 if wrong	PASS
18	New password min 6 chars enforced	Owner	400 if too short	PASS
19	PIN change only valid for staff role	Owner	400 for manager/owner	PASS
20	Staff can update own avatar	Staff	200	PASS

#	Test Case	Role	Expected	Result
21	Staff cannot update another user avatar	Staff	403	PASS
22	Unauthenticated requests rejected on all user routes	No token	401	PASS

4. Test Cases — Store Configuration

#	Test Case	Expected	Result
1	Owner updates store name and address	200 + updated store object	PASS
2	Manager cannot update store profile	403	PASS
3	Owner sets billing mode to ephemeral	200	PASS
4	Owner sets billing mode to structured	200	PASS
5	Invalid billing mode value rejected	400	PASS
6	Manager cannot change billing mode	403	PASS
7	Owner sets retention period in days	200	PASS
8	Retention period below 1 day rejected	400	PASS
9	Owner sets GPS coordinates and radius	200 + lat/lng/radius persisted	PASS
10	gpsRequireClockIn = true persisted to database	200 + confirmed in DB row	PASS
11	gpsRequireClockOut = true persisted to database	200 + confirmed in DB row	PASS
12	Zero GPS radius rejected	400	PASS
13	String values for coordinates rejected	400	PASS
14	Missing gpsLatitude rejected	400	PASS
15	Missing gpsLongitude rejected	400	PASS
16	Manager cannot update GPS settings	403	PASS
17	Staff cannot update GPS settings	403	PASS
18	Unauthenticated request rejected	401	PASS

5. Test Cases — Categories & Price Bands

Categories

#	Test Case	Expected	Result
1	Owner lists categories (includes seeded)	200 + array	PASS
2	Category shape: id, name, storeId, createdAt	All fields present	PASS
3	No cross-store category leakage	Only own-store categories	PASS
4	Owner creates category	201 + created object	PASS
5	Manager creates category	201	PASS
6	Staff cannot create category	403	PASS
7	Empty name rejected	400	PASS
8	Duplicate category name rejected	409	PASS
9	New category appears in subsequent GET	Persisted correctly	PASS
10	Category with active price bands cannot be deleted	Remains after failed delete	PASS

Price Bands

#	Test Case	Expected	Result
11	Owner creates price band under category	201 + band object	PASS
12	Price of 0 or negative rejected	400	PASS
13	categoryId from another store rejected	404	PASS
14	GET price bands with stock shows correct totals	200 + totalStock per band	PASS
15	Staff can read price bands	200	PASS
16	Staff cannot create price bands	403	PASS

6. Test Cases — Inventory Batches

#	Test Case	Expected	Result
1	Owner adds batch (20 units at seeded price band)	201 + quantityRemaining = quantityAdded	PASS
2	Manager adds batch	201	PASS
3	Staff cannot add batch	403	PASS
4	Batch without optional fields (vendor, cost, notes)	201 — optional fields nullable	PASS
5	quantityAdded = 0 rejected	400	PASS
6	Negative quantityAdded rejected	400	PASS
7	Float quantityAdded rejected (must be integer)	400	PASS
8	priceBandId from another store rejected	404	PASS
9	costPrice of zero or negative rejected	400	PASS
10	Newly added batch appears in GET /inventory/batches	Persisted correctly	PASS
11	filter=low_stock returns batches with qty < 10	200 filtered	PASS
12	filter=aging returns batches older than 60 days	200 (empty for fresh test data)	PASS
13	filter=new_arrivals returns batches within last 7 days	200 including seeded batch	PASS
14	Invalid filter value rejected	400	PASS
15	Margin computed when costPrice and price are present	margin field not null	PASS
16	Adjust quantity upward	200 + increased qty	PASS
17	Adjust quantity downward	200 + decreased qty	PASS
18	Adjust to 0 (full depletion)	200	PASS
19	Adjust with negative quantity rejected	400	PASS
20	Staff cannot adjust batch	403	PASS
21	Adjust reflected in subsequent GET	Updated qty in response	PASS
22	Close batch sets quantityRemaining to 0	200	PASS

#	Test Case	Expected	Result
23	Staff cannot close batch	403	PASS
24	Mark batch defective appends [DEFECTIVE] to notes	200 + notes updated	PASS
25	Defective appends to existing notes	[DEFECTIVE] added to existing text	PASS
26	Defective when notes null sets notes to [DEFECTIVE]	notes = "[DEFECTIVE]"	PASS
27	Defective action does not change quantityRemaining	qty unchanged	PASS
28	Invalid action value rejected	400	PASS
29	Non-existent batch id returns 404	404	PASS
30	Stock matrix has correct category / band / stock structure	200 + band "299" with stock 20	PASS
31	Stock values in matrix are non-negative integers	All values ≥ 0	PASS

7. Test Cases — Sales & Returns

Sales (POST /api/sales)

#	Test Case	Expected	Result
1	Staff creates a sale — response has id and createdAt	201	PASS
2	Owner creates sale with UPI payment	201	PASS
3	Manager creates sale with store_credit payment	201	PASS
4	FIFO: deducts from oldest batch first, spills to newer	201 + oldest batch depleted first	PASS
5	Insufficient stock returns descriptive error with item name	400	PASS
6	priceBandId with zero batches returns insufficient stock	400	PASS
7	Custom item (priceBandId: 0) — no stock deduction	201 + stock unchanged	PASS
8	Custom item mixed with normal item — only normal item deducted	201	PASS
9	New customer created when phone first seen	201 + customer record exists	PASS
10	Existing customer name updated on second sale with same phone	201 + name updated	PASS
11	Sale without customerPhone creates no customer record	201 + no customer inserted	PASS
12	discountAmount defaults to 0 when omitted	201 + discountAmount = 0	PASS
13	Negative discountAmount rejected	400	PASS
14	Invalid paymentMethod rejected	400	PASS
15	Empty items array rejected	400	PASS
16	Item missing categoryName rejected	400	PASS
17	Item quantity less than 1 rejected	400	PASS
18	Non-10-digit customerPhone rejected	400	PASS
19	No token returns 401	401	PASS

Returns & Exchanges (POST /api/returns)

#	Test Case	Expected	Result
20	Owner creates return for existing sale	201	PASS
21	Manager creates return	201	PASS
22	Staff creates return (no role restriction on POST)	201	PASS
23	Exchange type accepted	201	PASS
24	Full quantity return accepted	201	PASS
25	Stock restored to inventory after return	quantityRemaining increases	PASS
26	Full quantity return fully restores stock	Stock back to original level	PASS
27	Return quantity exceeds sold quantity	400	PASS
28	Non-existent saleId returns 404	404	PASS
29	Non-existent saleItemId returns 404	404	PASS
30	Returns list ordered newest first	createdAt descending	PASS
31	Staff cannot list returns	403	PASS
32	Missing type field rejected	400	PASS
33	Invalid type value rejected	400	PASS
34	Quantity of 0 rejected	400	PASS

8. Test Cases — Vendors & Reorders

Vendors

#	Test Case	Expected	Result
1	Owner creates vendor with name, phone, city	201 + vendor object	PASS
2	Vendor with name only (all optional fields omitted)	201	PASS
3	Vendor created with notes	201	PASS
4	Manager creates vendor	201	PASS
5	Staff cannot create vendor	403	PASS

#	Test Case	Expected	Result
6	Name too short (< 2 chars) rejected	400	PASS
7	Two vendors with same phone allowed (no unique constraint)	201	PASS
8	Owner updates vendor name	200 + updated	PASS
9	Manager updates vendor	200	PASS
10	Staff cannot update vendor	403	PASS
11	Owner soft-deletes vendor	200 + absent from GET	PASS
12	Deleting already-deleted vendor returns 404	404 (not 200)	PASS
13	Manager cannot delete vendor (owner-only)	403	PASS
14	Suggest-order returns per-band suggested quantities	200 + suggestedQty array	PASS
15	suggestedQty formula: $\max(0, \text{ceil}(\text{dailyAvg} \times 14) - \text{currentStock})$	Correct computed value	PASS
16	Suggest-order for non-existent vendor returns 404	404	PASS

Reorders

#	Test Case	Expected	Result
17	Owner creates reorder with vendorId and items	201 + reorder object	PASS
18	Manager creates reorder	201	PASS
19	Staff cannot create reorder	403	PASS
20	messageText generated on creation (vendor name + items)	Non-empty formatted string	PASS
21	Items with finalQty = 0 excluded from messageText	Zero-qty items absent	PASS
22	messageText preserved in GET /api/reorders	Same text accessible later	PASS
23	Reorder status updated to acknowledged	200 + status = acknowledged	PASS
24	Reorder status updated to fulfilled	200 + status = fulfilled	PASS
25	Reorder status updated to draft	200 + status = draft	PASS
26	Invalid status value rejected	400	PASS
27	Non-existent reorder id returns 404	404	PASS

#	Test Case	Expected	Result
28	Non-existent vendorId returns 404	404	PASS
29	Reorders list ordered by createdAt descending	Newest first	PASS

9. Test Cases — Attendance & GPS Geofencing

#	Test Case	Expected	Result
1	Staff clocks in — returns clockedIn: true and checkInAt	200	PASS
2	Duplicate clock-in on same day rejected	400 already clocked in	PASS
3	Owner can clock in (GPS not required by default)	200	PASS
4	GPS required + coords within radius: clock-in succeeds	200	PASS
5	GPS required + coords outside radius: clock-in rejected	400 geofence error	PASS
6	GPS not required + coords outside radius: clock-in accepted	200	PASS
7	Missing lat/lng fields rejected	400	PASS
8	Successful clock-out after clock-in — clockedIn: false	200 + checkOutAt present	PASS
9	Clock-out without prior clock-in rejected	400	PASS
10	Second clock-out rejected	400	PASS
11	GPS not required: clock-out with remote coords accepted	200	PASS
12	Attendance row records check_in_lat and check_out_lat in DB	Coordinates saved correctly	PASS
13	Staff cannot view today roster	403	PASS
14	Owner can view today roster	200 + array	PASS
15	Manager can view today roster	200	PASS
16	Manager can force clock-out a staff member	200	PASS
17	Unauthenticated clock-in rejected	401	PASS
18	Unauthenticated clock-out rejected	401	PASS

10. Test Cases — Dashboard, Staff Leaderboard & Audit

Dashboard

#	Test Case	Expected	Result
1		200 + KPI object	PASS

#	Test Case	Expected	Result
	Owner receives KPI summary with required fields		
2	Manager receives KPI summary	200	PASS
3	activeVendors count includes seeded vendor	count $\geq$ 1	PASS
4	Staff cannot access dashboard	403	PASS
5	Daily summary with ?range=weekly returns array	200	PASS
6	Daily summary with ?range=monthly returns array	200	PASS
7	Daily summary rows have expected shape	bandPrice, totalQtySold, totalRevenue present	PASS
8	Invalid range value rejected	400	PASS

Staff Leaderboard

#	Test Case	Expected	Result
9	Owner receives leaderboard array	200 + array	PASS
10	All active seeded users appear in leaderboard	3 entries (owner, manager, staff)	PASS
11	Leaderboard sorted by revenueToday descending	Correct sort order	PASS
12	salesCountToday defaults to 0 when no sales recorded	0	PASS
13	Leaderboard accessible to staff	200	PASS
14	Only active users from same store included	No cross-store or inactive entries	PASS

Audit Log & Ephemeral Wipe

#	Test Case	Expected	Result
15	Owner views audit log — empty on fresh DB	200 + empty array	PASS
16	Manager cannot view audit log	403	PASS
17	Staff cannot view audit log	403	PASS
18	Wipe with correct password prunes ephemeral sales	200 + recordsPruned > 0	PASS

#	Test Case	Expected	Result
19	Wipe with no ephemeral sales reports 0 pruned	200 + recordsPruned = 0	PASS
20	Wipe with wrong password rejected	401	PASS
21	Wipe entry appears in audit log afterwards	Log entry with type and timestamp	PASS
22	Manager cannot trigger wipe	403	PASS
23	Staff cannot trigger wipe	403	PASS
24	Missing password field rejected	400	PASS

11. Bugs Found and Fixed During Testing

Testing with Jest + Supertest directly revealed 7 critical defects before any manual QA was performed. All were resolved and the associated tests now pass.

BUG-01 — GPS Geofencing Completely Non-Functional

Affected files: `frontend/src/pages/StaffPage.tsx`, `backend/src/services/attendance.ts`

Symptom: Enabling "Require GPS on Clock-In" had no effect. Staff could clock in from any location regardless of the toggle state.

Root Cause: GPS policy flags were stored only in Zustand (browser memory) and never written to the database. The backend always read from an empty `gps_settings` table and therefore never enforced the policy.

FIX-01

Added `gps_require_clock_in` and `gps_require_clock_out` boolean columns to the `stores` table. Updated `PATCH /store/gps-settings` to persist both fields. Backend attendance service now reads flags directly from the store row. Frontend `StaffPage` calls the API on every toggle change.

BUG-02 — Returns Service: column "store_id" does not exist

Affected file: `backend/src/services/returns.ts`

Symptom: `POST /api/returns` always failed with PostgreSQL error: `column "store_id" does not exist`.

Root Cause: The SQL query tried to select `store_id` from the `sale_items` table, which has no such column.

FIX-02

Changed the query to pass `storeId` as the `$3` bind parameter rather than attempting to read it from `sale_items`.

BUG-03 — Returns Service: Inconsistent Types for Parameter \$1

Affected file: `backend/src/services/returns.ts`

Symptom: PostgreSQL rejected the query with:
`inconsistent types deduced for parameter $1`.

Root Cause: `$1` appeared in both an integer comparison and an arithmetic multiplication in the same query, causing the query planner to fail on type inference.

FIX-03

Added explicit PostgreSQL casts: `$1::int` for the WHERE clause and `$1::numeric` for multiplication.

BUG-04 — Vendor Soft-Delete: Second Delete Returns 200 Instead of 404

Affected file: `backend/src/services/vendors.ts`

Symptom: Deleting a vendor that was already soft-deleted returned HTTP 200 instead of 404.

Root Cause: `UPDATE ... SET is_active = FALSE WHERE id = $1` matched the row regardless of its current `is_active` value, so PostgreSQL still reported `rowsAffected = 1`.

FIX-04

Added `AND is_active = TRUE` to the WHERE clause. A second delete now results in `rowsAffected = 0` and the service correctly returns 404.

BUG-05 — Inventory Matrix Test: Hardcoded Stock Assertion Fails Intermittently

Affected file: `backend/src/__tests__/inventory.test.ts`

Symptom: Test "bands object contains the seeded price (299) with correct stock (20)" failed with `expected 20, received 22`.

Root Cause: An earlier test in the same suite added 2 units to trigger the low-stock filter threshold, leaving the seeded batch at 22 units by the time the matrix test ran.

FIX-05

Changed `expect(stockAt299).toBe(20)` to `expect(stockAt299).toBeGreaterThanOrEqual(20)`, which correctly asserts stock is present without assuming within-suite isolation.

BUG-06 — Jest 30 + ts-jest 29 Incompatibility: All Tests Fail on Import

Affected files: `backend/jest.config.ts`, `backend/package.json`

Symptom: All 480 tests failed immediately with `Cannot find module '../..app'` and `Cannot find module '../helpers/db'`.

Root Cause: Jest 30 changed internal module resolution APIs incompatible with ts-jest 29. Wrong relative import paths in test files compounded the issue.

FIX-06

Pinned Jest to `^29.7.0`. Updated `jest.config.ts` to use the explicit `transform` key. Fixed all relative import paths: `../..app` → `../app`, `../helpers/db` → `./helpers/db`, and corrected the schema SQL path (one directory level off).

BUG-07 — node_modules Committed to Git Repository

Affected file: Root `.gitignore` (missing)

Symptom: The repository tracked `backend/node_modules/`, bloating repository size by hundreds of megabytes and making all git operations extremely slow.

Root Cause: No `.gitignore` file existed at the repository root.

FIX-07

Created root-level `.gitignore` excluding `node_modules/`, `dist/`, `.env`, and `*.env.local`. Ran `git rm -r --cached backend/node_modules` to remove tracked files without deleting them locally.

12. User Feedback

Feedback was collected from the primary user interview (Bombay Fashion) and internal team review sessions during development.

BUG REPORT

GPS Distance Shows Absurd Value (e.g., 393,191 m from store) — Story 7.1

The ClockWidget displayed "393191m from store." Store GPS coordinates were at the PostgreSQL default of 0, 0 (Gulf of Guinea), so any real device produced a huge haversine distance.

Resolution: Store coordinates updated to actual location. ClockWidget now hides the distance pill when coordinates are 0, 0.

BUG REPORT

GPS Enforcement Toggles Have No Effect — Story 7.1

Enabling "Require GPS on Clock-In" did nothing. Staff could clock in from anywhere. **Resolution:** Toggles now persist to the database via `PATCH /store/gps-settings` and the backend enforces the flags on every clock-in/out request.

UX FEEDBACK

API Documentation Contained Emojis — Documentation Quality

The OpenAPI YAML description used emoji section headers inconsistent with the professional tone of the Milestone 1 submission. **Resolution:** Entire description rewritten using the exact 8 EPICs and 20 user stories from the Milestone 1 PDF in proper "As a... I want... So that..." format without emojis.

FEATURE GAP

WhatsApp Reorder Message Not Auto-Sent — Story 5.2

The system generates a well-formatted reorder message but requires the manager to manually copy it into WhatsApp. **Status:** Planned for Sprint 2. Milestone 1 spec notes "generic messaging/email integration" due to WhatsApp Business API cost and approval requirements.

UX FEEDBACK

Billing Mode Switch Has No Confirmation Step — Story 3.2

Switching from Structured to Ephemeral billing mode is a high-impact action but the UI applies it immediately without a confirmation dialog. **Status:** Planned for Sprint 2.

FEATURE GAP

No Barcode / QR Scan for Price Band Selection on Mobile — Story 3.1

Staff must scroll a dropdown to find the correct price band during billing. A scan shortcut was requested for faster shop-floor entry. **Status:** Planned for Sprint 3 pending `BarcodeDetector` API availability on Android Chrome.

PERFORMANCE

Dashboard Summary Has No Caching — Story 8.1

`GET /api/dashboard/summary` executes multiple aggregation queries on every request. Response times may degrade with large sales history. **Status:** 5-minute in-memory cache planned for Sprint 2.

13. Sprint 2 — Planned Improvements

Item	Category	Story	Description	Priority
Dashboard caching	Performance	8.1, 8.2	5-minute in-process cache for <code>/dashboard/summary</code> and <code>/dashboard/daily-summary</code> to reduce repeated aggregation queries under load.	High
Billing mode confirmation modal	UX / Safety	3.2	Add a confirmation dialog before switching to Ephemeral mode, warning the owner that future sales will not be permanently retained.	High
Reorder messaging integration	Feature	5.2	Integrate a generic email/SMS gateway so "Send Reorder" dispatches the generated message text directly without requiring manual copy-paste.	High
Automated end-of-day clock-out	Feature	7.1	Scheduled job at 23:59 to force clock-out any staff still marked as clocked in, preventing orphaned attendance records.	Medium
Weekly staff sales summary endpoint	Feature	7.2	<code>GET /staff/summary?period=weekly</code> returning per-staff revenue and units sold for the configured period. Current leaderboard covers only the current calendar day.	Medium
Discount recommendation field	Feature	8.3	Add <code>suggestedDiscountPct</code> to aging batch responses, computed from batch age, cost margin, and current stock level.	Medium
Repeat purchase metric endpoint	Feature	6.2	<code>GET /customers/repeat-rate?days=30</code> returning the percentage of customers with more than one purchase in the window.	Medium
Filter-based customer export	Feature	6.3	Query parameters on <code>GET /customers</code> to filter by price band purchased and date range for targeted outreach list exports.	Medium
Text-based trend insight summaries	Feature	8.2	<code>GET /dashboard/insights</code> returning auto-generated week-over-week text summaries such as "350 band sales increased 18% vs last week."	Medium
Barcode scan for price band selection	UX	3.1	Investigate <code>BarcodeDetector</code> API on Android Chrome for the mobile billing screen as an alternative to the price band dropdown.	Low

Item	Category	Story	Description	Priority
Rate limiting on auth endpoints	Security	1.1, 1.2	10 attempts per minute per IP on <code>POST /auth/login</code> and <code>POST /auth/register</code> to mitigate brute-force attacks.	Low
Pagination on list endpoints	Performance	All lists	<code>?page</code> and <code>?limit</code> parameters on <code>/inventory/batches</code> , <code>/sales</code> , and <code>/customers</code> to prevent unbounded query results.	Low

14. Test Infrastructure

Technology Stack

Component	Technology	Version
Test runner	Jest	29.7.0
TypeScript transformer	ts-jest	29.x
HTTP integration testing	Supertest	7.x
Test database	PostgreSQL	16 (smallbiz_test)
PostgreSQL client	node-postgres (pg)	8.x
Password hashing	bcrypt	5.x
Runtime	Node.js	22.x

Database Isolation Strategy

Each test suite calls `resetDb()` in its `beforeAll` block. This re-runs `database/schema.sql`, which drops all tables in reverse dependency order and recreates them from scratch. `seedBase()` then inserts a consistent set of test fixtures: one store, one owner, one manager, one staff member, one category, one price band, one vendor, and one inventory batch of 20 units. All test suites start from an identical known state.

Tests are run **serially** using `--runInBand` to prevent concurrent `resetDb()` calls racing on the shared test database. The npm test script is: `jest --runInBand --forceExit`.

Test Execution Command

From the `backend/` directory:

```
npm test → jest --runInBand --forceExit
```


All Test Files

File	Tests	Coverage Area
src/__tests__/auth.test.ts	27	Registration, login, JWT
src/__tests__/users.test.ts	67	CRUD, passwords, PINs, avatars, RBAC
src/__tests__/store.test.ts	41	Profile, billing mode, retention, GPS
src/__tests__/categories.test.ts	23	Category CRUD, uniqueness
src/__tests__/priceBands.test.ts	32	Price band CRUD, stock queries
src/__tests__/inventory.test.ts	62	Batches, matrix, adjust, defective, close
src/__tests__/sales.test.ts	22	FIFO deduction, customers, custom items
src/__tests__/returns.test.ts	30	Returns, exchanges, stock restoration
src/__tests__/vendors.test.ts	45	Vendor directory, soft-delete, suggest-order
src/__tests__/reorders.test.ts	38	Create, status updates, message text
src/__tests__/attendance.test.ts	20	Clock-in/out, GPS geofencing, roster
src/__tests__/staff.test.ts	12	Leaderboard, RBAC, sorting
src/__tests__/dashboard.test.ts	15	KPI summary, daily charts
src/__tests__/audit.test.ts	19	Audit log, ephemeral data wipe
TOTAL	480	All 33 API endpoints covered